

Course project: Solving forward and inverse problems in mathematical modeling of wave propagation

Contributed by Dr. Adar Kahana
Division of Applied Mathematics, Brown University

Project statement

Wave propagation is a widely researched topics mainly due to the various physical applications. The one-dimensional second order wave problem is given by:

$$\begin{cases} u_{tt}(x, t) = c^2(x)u_{xx}(x, t) & x \in [l_1, l_2], t \in (0, T] \\ u(x, 0) = u_0(x) & x \in [l_1, l_2] \\ u_t(x, 0) = v_0(x) & x \in [l_1, l_2] \\ u(x, t) = f(x, t) & x \in \Omega_1, t \in [0, T] \\ \frac{\partial u}{\partial x}(x, t) = g(x, t) & x \in \Omega_2, t \in [0, T] \end{cases} \quad \Omega_1 \cup \Omega_2 = \{l_1, l_2\}, \Omega_1 \cap \Omega_2 = \{\} \quad (1)$$

where $u(x, t)$ is the wave amplitude or acoustic pressure. For given initial conditions (u_0, v_0) and boundary conditions (f, g) , for types Dirichlet and Neumann respectively) the wave problem (1) is well-posed (small perturbations of the initial conditions result in small perturbations in the solution).

In this project, we will start by implementing a solver for the wave problem based on off-the-shelf numerical schemes and analyse the behavior of the wave propagation. We will then use the solver to produce data for training a physically-informed neural network (PINN) for approximating the forward and inverse wave propagation experiments. Next, we will move to the ill-posed problem of using partial data - recordings in a set of sensors.

Tasks

1. Find an analytic solution for the wave problem (1) in the following setup:

- The wave velocity $c = 1$ and the domain $\Omega = [0, 1]$.
- The boundary conditions are of type Dirichlet and set to 0: $u(0, t) = u(1, t) = 0$.
- The initial condition is an impulse of height 1 centered at $x = 0.5$:

$$u_0(x) = \begin{cases} 0 & \text{if } 0 \leq x < 0.45, \\ 20(x - 0.45) & \text{if } 0.45 \leq x < 0.5, \\ 20(0.55 - x) & \text{if } 0.5 \leq x < 0.55, \\ 0 & \text{if } 0.55 \leq x \leq 1. \end{cases}$$

You may use d'Alembert solution, separation of variables, etc. Use Python or MATLAB to produce the solution in a uniform grid where $\Delta x = 10^{-2}$ and $\Delta t = \frac{0.1\Delta x}{c}$ (the Courant–Friedrichs–Lewy number is 0.1). Use 800 or more time-steps to see a full reflection of the wave after the impulse reaches the boundary.

2. Implement a time marching numerical scheme such as finite-difference (forward Euler, central difference, etc) to approximate the solution of the problem from the previous task. Compare the results of the numerical approximation to the analytic solution: create a plot of the difference in the L_2 -norm between the approximated and the analytic solution over time.
3. Use a PINN to solve the problem in the first task. As input data, create the points and the pressures using the analytic solution you found earlier. There are several implementations of PINNs in Tensorflow 1 [1, 2, 3], TensorFlow 2 [1, 2] and PyTorch [2]. Create a similar plot to the one in the second tasks, now with two lines, adding a comparison between the PINN and the analytic solution.

4. *Using sensors*: from the analytic solution extract the points and pressures in 19 evenly spread locations on the x -axis: $x = 5\Delta x, x = 10\Delta x, \dots, x = 1 - 5\Delta x$. Use the pressures in these points for all time steps. The data-set for training the PINN consists of $\{u(x_k, t_n)\}_{k=1, n=1}^{19, N_t}$, where N_t is the total number of time steps. Plot the output of the PINN at time $t = 0$ and see if you can easily locate the initial impulse at the correct place $x = 0.5$.
5. Instead of using evenly spread sensors, use randomly spread sensors. Then, change the number of sensors and find a minimal number of sensors that are able to locate the initial impulse. Make sure your results are consistent (not a lucky choice of sensors placement).

Programming Options

You may use TensorFlow, PyTorch, or Modulus for completing the tasks.

References

- [1] A. D. Jagtap and G. E. Karniadakis. Extended physics-informed neural networks (xpinns): A generalized space-time domain decomposition based deep learning framework for nonlinear partial differential equations. *Communications in Computational Physics*, 28(5):2002–2041, 2020.
- [2] L. Lu, X. Meng, Z. Mao, and G. E. Karniadakis. DeepXDE: A deep learning library for solving differential equations. *SIAM Review*, 63(1):208–228, 2021.
- [3] M. Raissi, P. Perdikaris, and G. E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.